

Overlap removal

Overlap removal snaps first, then left-packs each row without changing row assignment

The idea

- Sort movables by x within each row and place left without overlap
- D, E, and F keep their seed roles on other rows
- This is the same shelf pack Tetris uses, simple, deterministic

Pseudocode

- Overlap removal is a two-phase sketch: snap first, then pack each row
- Pseudocode makes the phases explicit so you do not merge them into one vague “fix
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- Inside each row
- On the teaching seed that yields A at four

Algorithm sketch — try these

```
INPUT: positions, widths, rows, fixed macros
OUTPUT: legal packing (if capacity allows)
snap all movables to sites/rows
for each row y:
    sort movables by x; left-pack (skip macros)
report legal?, disp, HPWL
GOLDEN: A@4 B@6 C@8 on y=2; disp=6; HPWL=32
```

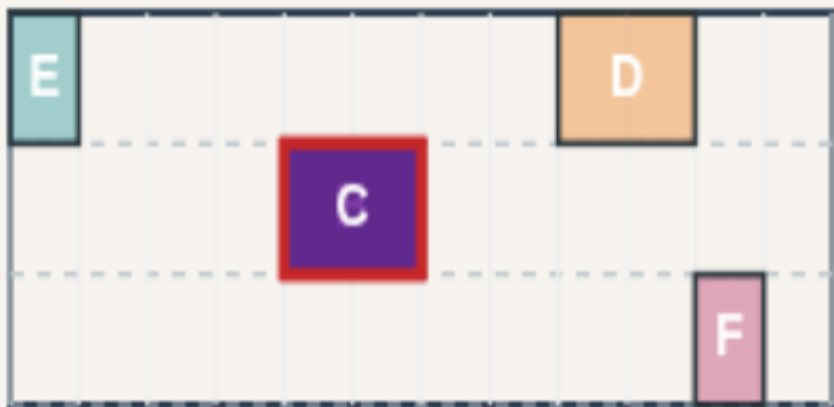
A, B, C stacked at (4, 2)

OVERLAP REMOVAL

A, B, C stacked at (4, 2)

Step 1 / 5 · overlap-seed · module02-03-overlap-removal

chip 12x6 · 3 rows · site 1



Explanation

The overlap seed piles three width-two cells on the middle row at x equals four. Greedy snap would leave the same conflict—overlap removal snaps first, then packs each row left-to-right.

- A,B,C @ (4,2)
- illegal: overlap A/B
- D@8,4 · E@0,4 · F@10,0

legal: false
reason: overlap A/B

A, B, C stacked at (4, 2)

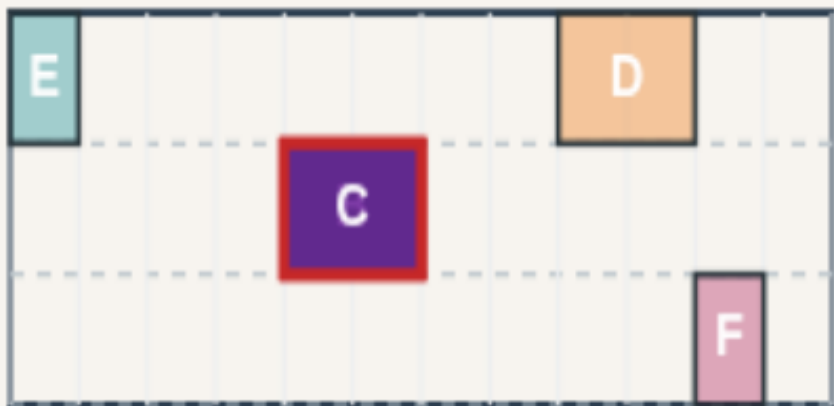
After snap: still stacked

OVERLAP REMOVAL

After snap: still stacked

Step 2 / 5 · after-snap · module02-03-overlap-removal

chip 12×6 · 3 rows · site 1



Explanation

Snapping the overlap seed does not separate A, B, and C—they remain on (4, 2). Overlap removal keeps their row assignment and resolves x conflicts by packing.

- Snap preserves row intent
- Overlap is an x problem on-row
- Pack by increasing x order

snap legal: false
middle row crowded

After snap: still stacked

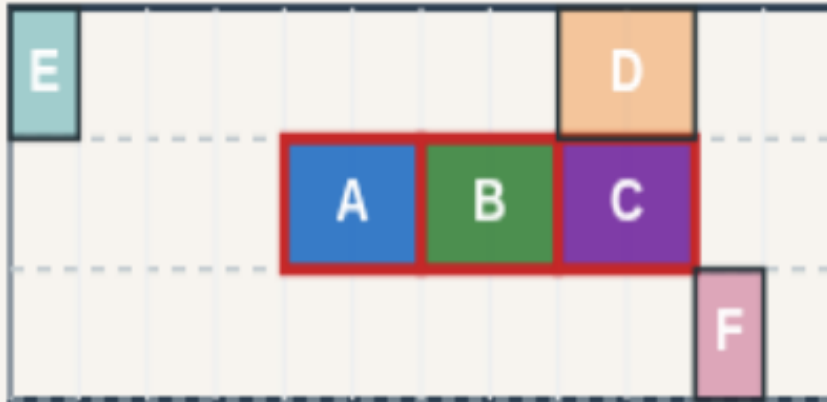
Per-row pack: A@4, B@6, C@8

OVERLAP REMOVAL

Per-row pack: A@4, B@6, C@8

Step 3 / 5 · row-pack · module02-03-overlap-removal

chip 12×6 · 3 rows · site 1



Explanation

On row y equals two, sort by x and place left without overlap: A at four, B at six, C at eight. Each width-two cell occupies two consecutive sites—now legal on that row.

- Preserve sort order from snap
- Left pack within row
- No cross-row moves

A @ (4,2)
B @ (6,2)
C @ (8,2)

Per-row pack: A@4, B@6, C@8

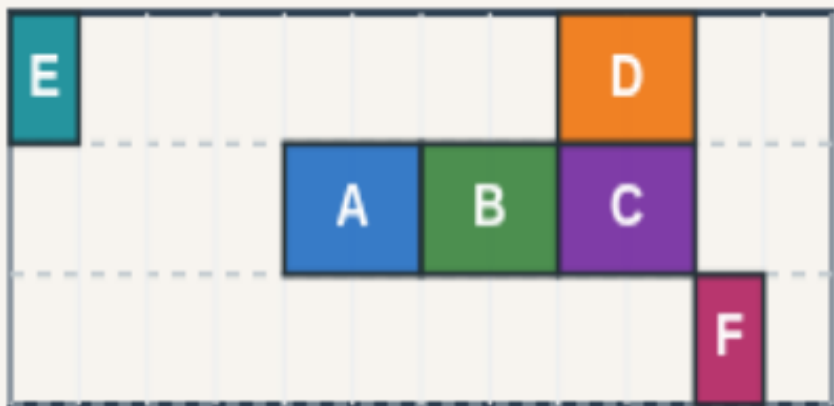
Legal: disp 6, HPWL 32

OVERLAP REMOVAL

Legal: disp 6, HPWL 32

Step 4 / 5 · legal-metrics · module02-03-overlap-removal

chip 12×6 · 3 rows · site 1



Explanation

Full overlap removal on the seed is legal with total displacement six and HPWL thirty-two. D, E, and F never moved from their overlap-seed roles.

- legal: true
- L1 disp from overlap origin
- Report HPWL with same nets

disp: 6
HPWL: 32
measured disp: 6

Legal: disp 6, HPWL 32

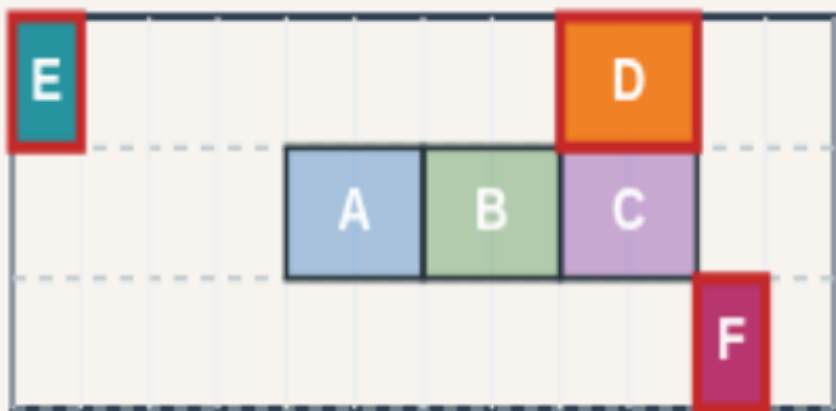
D, E, F unchanged

OVERLAP REMOVAL

D, E, F unchanged

Step 5 / 5 · unchanged · module02-03-overlap-removal

chip 12x6 · 3 rows · site 1



Explanation

Macro-sized D stays at (8, 4), E at (0, 4), F at (10, 0).
Overlap removal only repacked the crowded middle row —fixed-looking cells on other rows are untouched.

- D top row @ 8
- E top row @ 0
- F bottom row @ 10

movables repacked: A,B,C
others held row

D, E, F unchanged

Browser lab track

- In the browser lab track, open the **overlap-removal** lab from the tools shelf
- Open the interactive lab
- Reveal golden is study-only
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse `tiny\_legal.json`, run the algorithm with deterministic coordinates
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module